# SAHAYAK

## STATIC SIGN COLLECTION, TRAINING AND DEPLOYMENT

A repeatable step-by-step workflow for adding one static sign while preserving all existing classes.

### THE SAFE WORKFLOW

1. Back up the dataset and active model.
2. Collect the new sign into the existing CSV.
3. Verify all old and new labels.
4. Train a candidate, run tests, then promote only if correct.
5. Copy only the promoted static model to the deployment project.

**Static training uses data/landmarks_2hand.csv and 126 features.**

### Project folder

`C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training`

Do not train on only the new sign. The candidate must include all retained old classes.

# STEP 1 - BACK UP

Open PowerShell in VS Code and run:

```
Set-Location "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
.\.venv\Scripts\Activate.ps1
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\static_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
Copy-Item .\data\landmarks_2hand.csv $backup -Force
Copy-Item .\model\sahayak_twohand.joblib $backup -Force
Copy-Item .\model\labels_twohand.json $backup -Force
```

Confirm the prompt begins with (.venv). Avoid reinstalling working packages.

## Check current labels and counts

```
@'
import csv
from collections import Counter
with open("data/landmarks_2hand.csv", newline="", encoding="utf-8-sig") as f:
rows = list(csv.DictReader(f))
print("Rows:", len(rows))
print(Counter(r["label"] for r in rows))
'@ | python -
```

**Stop if the dataset is empty or an old label unexpectedly disappears.**

# STEP 2 - COLLECT

Run the existing collector:

```
python collect_data.py
```

Follow the collector prompts. Use one consistent label, usually uppercase with underscores, such as PLEASE.

Make sure the camera view clearly shows the complete hand pose. Capture varied attempts with natural changes in position and lighting.

## Verify the CSV after collection

```
@'
import csv
from collections import Counter
with open("data/landmarks_2hand.csv", newline="", encoding="utf-8-sig") as f:
rows = list(csv.DictReader(f))
print("Total:", len(rows))
for k,v in sorted(Counter(r["label"].strip().upper() for r in rows).items()):
print(k, v)
'@ | python -
```

## Before training

Confirm the CSV has the label column plus v0 through v125 (126 feature values).

Confirm every old intended label and the new label are present; review unexpected labels.

Collect enough varied samples; avoid saving many identical frames. Follow the collector's prompts.

# STEP 3 - TRAIN CANDIDATE

Train without promotion first:

```
python train_model.py
```

The trainer should create candidate files, leaving the active model unchanged:

```
model\sahayak_twohand_candidate.joblib
model\labels_twohand_candidate.json
```

## Inspect candidate labels and feature size

```
Get-Content .\model\labels_twohand_candidate.json
python -c "import joblib; m=joblib.load(r'model\sahayak_twohand_candidate.joblib');
print('Features:',m.n_features_in_); print('Labels:',sorted(str(x) for x in m.classes_))"
```

Expect 126 features and all intended old labels plus the new label.

Do not promote if any intended old class is missing or an accidental label appears.

## Run tests

```
python -m unittest test_sahayak.py
```

A perfect random holdout score is encouraging, but it does not prove real-camera accuracy. Test fresh captures.

# STEP 4 - PROMOTE AND TEST

Only after candidate labels and tests are correct, activate the candidate:

```
python train_model.py --promote
```

Check that the active model and labels were updated successfully. Keep the backup path printed by the trainer.

## Verify active labels

```
Get-Content .\model\labels_twohand.json
python -c "import joblib; m=joblib.load(r'model\sahayak_twohand.joblib'); print(m.n_features_in_,
sorted(str(x) for x in m.classes_))"
```

## Test local camera

```
python web_app.py
```

Try the new sign and several old signs in the local website. Check the label, landmarks/box and stability.

**If old signs regress, restore the backed-up model and labels before publishing.**

# STEP 5 - PUBLISH STATIC MODEL

Copy only the promoted static model into the deployment project:

```
.\deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_twohand.joblib
```

If labels_twohand.json is maintained as metadata, copy the promoted labels file to the backend root as well.

Commit the changed model file(s) to the branch watched by Render. Wait for the deployment to finish.

## Verify the backend

```
https://sahayak-backend-krh1.onrender.com/api/status
```

Check model_ready is true and single_sign_labels includes the new sign. Then test the live Studio.

## Safety checklist

[ ] Backup created. [ ] Existing labels retained. [ ] Candidate inspected.

[ ] Tests pass. [ ] Old and new signs tested locally. [ ] Correct model deployed.

Never upload the raw CSV/database unless there is a specific need. Never replace the sequence model when publishing static signs.

**If a command fails, stop before promotion and save the complete error output.**

# COPY/PASTE COMMANDS - STATIC SIGNS (1/3)

Run each shaded block in VS Code PowerShell. Paste one block at a time. The path matches this guide; change only the first line if your folder is elsewhere.

### 1. Open the project and activate Python

```
$ProjectRoot = "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
if (-not (Test-Path $ProjectRoot)) { throw 'Edit $ProjectRoot to the folder on this PC.' }
Set-Location $ProjectRoot
if (-not (Test-Path ".\.venv\Scripts\Activate.ps1")) { throw ".venv is missing." }
.\.venv\Scripts\Activate.ps1
python --version
python -c "import sys; print(sys.executable)"
```

### 2. Back up the CSV and active static-sign model

```
if (-not (Test-Path ".\data\landmarks_2hand.csv")) { throw "Dataset CSV missing." }
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\static_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
Copy-Item ".\data\landmarks_2hand.csv" $backup -ErrorAction Stop
Copy-Item ".\model\sahayak_twohand.joblib" $backup -ErrorAction Stop
if (Test-Path ".\model\labels_twohand.json") { Copy-Item ".\model\labels_twohand.json" $backup -Force }
Write-Host "Backup saved to $backup"
```

### 3. Check the current dataset and all label counts

```
@'
import csv
from collections import Counter
from pathlib import Path
path = Path("data/landmarks_2hand.csv")
if not path.exists():
    raise SystemExit("Dataset CSV not found; check the project folder.")
with path.open(newline="", encoding="utf-8-sig") as f:
    rows = list(csv.DictReader(f))
if not rows:
    raise SystemExit("Dataset is empty. Stop before training.")
counts = Counter(row["label"].strip().upper() for row in rows if row.get("label", "").strip())
print("Rows:", len(rows))
print("Label counts:")
for label, count in sorted(counts.items()):
    print(f"{label}: {count}")
print("All v0-v125 columns present:", all(f"v{i}" in rows[0] for i in range(126)))
'@ | python -
```

The dataset must keep all old labels as well as the new label. Stop if any intended class is missing or the CSV columns are wrong.

# COPY/PASTE COMMANDS - STATIC SIGNS (2/3)

### 4. Collect the new static pose

```
python collect_data.py
```

Follow the prompts and use one consistent label, usually uppercase with underscores (for example PLEASE). Make sure the hand pose is clearly visible. Vary position and lighting naturally.

### 5. Recheck rows and labels after collection

```
@'
import csv
from collections import Counter
with open("data/landmarks_2hand.csv", newline="", encoding="utf-8-sig") as f:
    rows = list(csv.DictReader(f))
if not rows:
    raise SystemExit("Dataset is empty. Stop before training.")
print("Rows:", len(rows))
for label, count in sorted(Counter(r["label"].strip().upper() for r in rows).items()):
    print(f"{label}: {count}")
'@ | python -
```

### 6. Train a candidate without changing the active model

```
python train_model.py
```

### 7. Confirm candidate feature count and retained labels

```
@'
import joblib
model = joblib.load(r"model\sahayak_twohand_candidate.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -
```

### 8. Run automated tests before promotion

```
python -m unittest test_sahayak.py
```

Expect 126 features, all intended old labels plus the new one, and passing tests. Do not promote a candidate that drops an old class.

# COPY/PASTE COMMANDS - STATIC SIGNS (3/3)

### 9. Promote only after reviewing candidate labels and tests

```
python train_model.py --promote
```

### 10. Verify the active static-sign model

```
@'
import joblib
model = joblib.load(r"model\sahayak_twohand.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -
```

### 11. Run the local app and test old plus new signs

```
python web_app.py
```

Keep this terminal open while testing the local website. Press Ctrl+C in this terminal to stop the local app after testing.

### 12. Copy only the promoted static model into the deployment project

```
$deployModel = ".\deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_twohand.joblib"
if (-not (Test-Path (Split-Path $deployModel))) { throw "Deployment model folder not found; confirm its path." }
Copy-Item ".\model\sahayak_twohand.joblib" $deployModel -Force
Write-Host "Copied model to $deployModel"
```

### 13. Check the live backend after the Render deployment finishes

```
Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status" | ConvertTo-Json -Depth 5
```

Confirm model_ready is true and single_sign_labels contains the new sign. The backend reads labels from the promoted model. Do not upload the raw CSV or training database.